

Corso di Programmazione Web

Lezione 1 — Storia e architettura del WWW

```
/**  
 * @author Rocco Mazzeo  
 * @email rocco.mazzeo@gmail.com  
 * @linkedin https://www.linkedin.com/in/roccomazzeo  
 */
```

Origini del World Wide Web

- Il World Wide Web nasce nel 1989 al CERN grazie a Tim Berners-Lee
- Obiettivo: condividere informazioni tra ricercatori tramite ipertesto
- Primi standard: HTML, HTTP, URL
- Il primo sito web: info.cern.ch (1991)
- Rapida diffusione negli anni '90 con Mosaic e Netscape

Concetti di client, server, protocollo HTTP

- **Client:** il programma che richiede risorse (es. browser)
- **Server:** il programma che fornisce risorse (es. web server Apache)
- **HTTP (HyperText Transfer Protocol):** protocollo di comunicazione tra client e server
- Funzionamento base:
 - i. Il client invia una richiesta HTTP (GET, POST, ...)
 - ii. Il server risponde con una risorsa (pagina HTML, immagine, ...)
- HTTP è senza stato (stateless): ogni richiesta è indipendente

Approfondimento: Protocollo HTTP

- HTTP (HyperText Transfer Protocol) è il protocollo fondamentale per la comunicazione sul Web
- Basato su un modello richiesta/risposta tra client e server
- Utilizza porte standard: 80 (HTTP), 443 (HTTPS)
- Le versioni principali: HTTP/1.0, HTTP/1.1, HTTP/2, HTTP/3

La pila ISO/OSI e il posizionamento di HTTP

La pila ISO/OSI (Open Systems Interconnection) è un modello a 7 livelli che descrive come i dati viaggiano in una rete:

1. **Fisico:** trasmissione dei bit su cavi o onde radio
2. **Collegamento dati:** trasferimento affidabile tra due nodi adiacenti (es. Ethernet)
3. **Rete:** instradamento dei pacchetti tra reti diverse (es. IP)
4. **Trasporto:** comunicazione affidabile tra host (es. TCP, UDP)
5. **Sessione:** gestione delle sessioni di comunicazione
6. **Presentazione:** traduzione, cifratura, compressione dei dati
7. **Applicazione:** servizi di rete per le applicazioni (es. HTTP, FTP, SMTP)

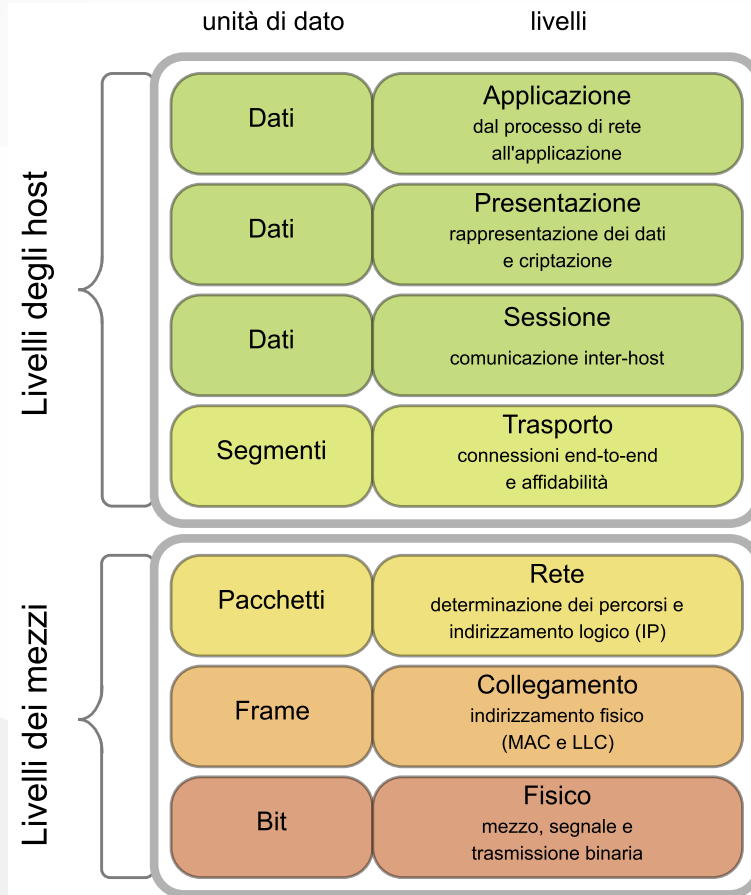


Figura: Rappresentazione grafica dei 7 livelli del modello ISO/OSI. HTTP si trova al livello Applicazione (7).

HTTP opera al **livello 7 (Applicazione)** del modello OSI. Si appoggia tipicamente su TCP (livello 4) e IP (livello 3). La pila reale di protocolli per una richiesta HTTP è quindi:

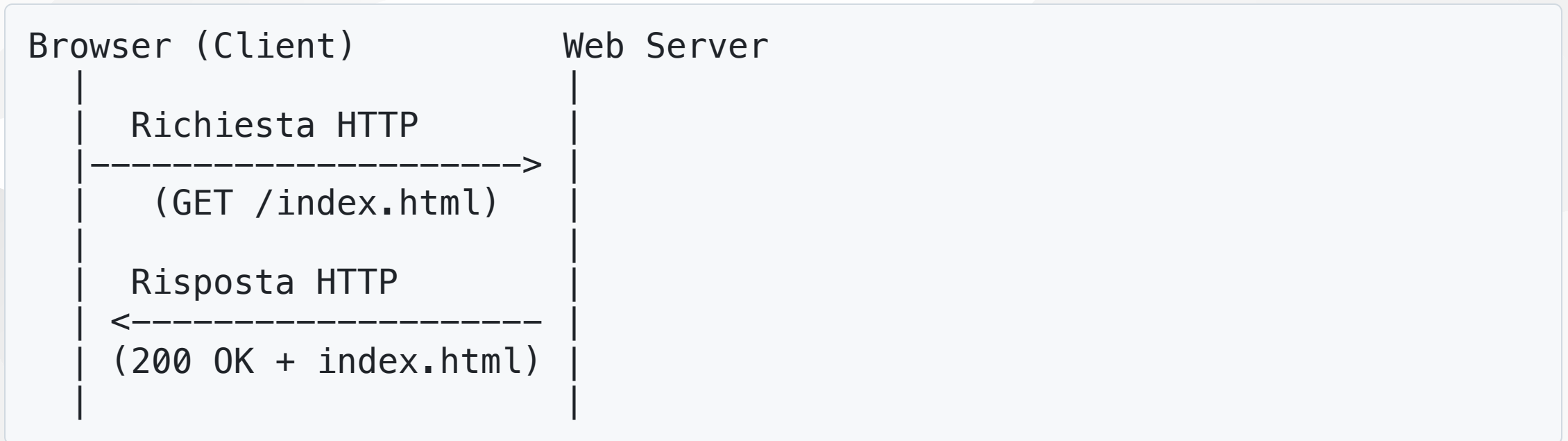
- Applicazione: **HTTP**
- Trasporto: **TCP**
- Rete: **IP**
- Collegamento dati: Ethernet/Wi-Fi, ecc.
- Fisico: cavo, fibra, onde radio, ecc.

Questo posizionamento permette a HTTP di concentrarsi sulla comunicazione tra applicazioni, delegando la trasmissione dei dati ai livelli inferiori.

Struttura di una richiesta HTTP

- Una richiesta HTTP è composta da:
 - **Metodo:** indica l'azione (GET, POST, PUT, DELETE, ecc.)
 - **URL:** risorsa richiesta
 - **Header:** informazioni aggiuntive (User-Agent, Accept, ecc.)
 - **Body:** dati inviati (solo in alcuni metodi, es. POST)

Esempio di interazione client-server:



Questo schema mostra come il browser invia una richiesta HTTP al server e riceve una risposta con la risorsa richiesta.

Strumenti per testare le API: Postman

- **Postman** è uno strumento grafico molto usato per testare e sviluppare API web.
- Permette di inviare richieste HTTP (GET, POST, ecc.), vedere le risposte, analizzare header e body, e automatizzare test.
- Utile sia per sviluppatori backend che frontend.

Esempio: richiesta GET a una pagina pubblica con Postman

1. Apri Postman e crea una nuova richiesta.
2. Seleziona il metodo **GET**.
3. Inserisci l'URL della risorsa, ad esempio:
`https://www.google.com`
4. Premi **Send**.

Risultato atteso:

Riceverai una risposta HTTP con status code `200 OK` e il body con il codice HTML della homepage di Google.

```
GET / HTTP/1.1
Host: www.google.com
User-Agent: PostmanRuntime/7.32.2
Accept: */*
...

HTTP/1.1 200 OK
Content-Type: text/html; charset=ISO-8859-1
...
<html>...</html>
```

- Puoi visualizzare header, status code e contenuto della risposta direttamente nell'interfaccia di Postman.
- Postman permette anche di salvare richieste e creare collezioni di test.

Esercizio: Visualizzare richiesta e risposta HTTP con Postman

1. Apri Postman e crea una nuova richiesta.
2. Seleziona il metodo **GET**.
3. Inserisci l'URL:

```
https://jsonplaceholder.typicode.com/posts/1
```

4. Premi **Send**.

Osserva:

- Nella scheda **Request** puoi vedere i dettagli della richiesta inviata (metodo, URL, header).
- Nella scheda **Response** puoi vedere lo status code, gli header e il body della risposta.

Esempio di risposta:

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
...

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit..."
}
```

Struttura di una risposta HTTP

- Una risposta HTTP contiene:
 - **Status code:** codice di stato (200 OK, 404 Not Found, 500 Internal Server Error, ecc.)
 - **Header:** informazioni sulla risposta (Content-Type, Set-Cookie, ecc.)
 - **Body:** contenuto della risposta (pagina HTML, JSON, file, ecc.)

Status Code

Status Code	Significato	Descrizione breve
200	OK	Richiesta eseguita con successo
301	Moved Permanently	Risorsa spostata in modo permanente
404	Not Found	Risorsa non trovata
500	Internal Server Error	Errore interno del server

Header

Header	Descrizione	Esempio
Content-Type	Tipo di contenuto restituito	text/html, application/json
Set-Cookie	Imposta cookie nel browser	sessionid=abc123; Path=/ 348
Content-Length	Dimensione del body in byte	348
Location	URL di reindirizzamento (usato con 3xx)	https://esempio.com/nuovo

Body

Body	Descrizione	Esempio
HTML	Pagina web visualizzabile dal browser	<code><html>...</html></code>
JSON	Dati strutturati per applicazioni web	<code>{ "nome": "Mario" }</code>
File	Qualsiasi tipo di file (immagine, PDF, ecc.)	(binario o base64)

Elenco completo degli HTTP Status Code

Gli status code HTTP sono suddivisi in 5 classi principali:

1xx — Informational

Codice	Significato	Descrizione breve
100	Continue	Richiesta ricevuta, continuare
101	Switching Protocols	Cambio di protocollo richiesto
102	Processing	Elaborazione in corso (WebDAV)
103	Early Hints	Informazioni preliminari

2xx — Success

Codice	Significato	Descrizione breve
200	OK	Richiesta eseguita con successo
201	Created	Risorsa creata
202	Accepted	Richiesta accettata, in elaborazione
203	Non-Authoritative Information	Risposta da fonte non autorevole
204	No Content	Nessun contenuto da restituire
205	Reset Content	Richiesta di reset della vista
206	Partial Content	Risposta parziale (range)
207	Multi-Status	Risposte multiple (WebDAV)
208	Already Reported	Già riportato (WebDAV)
209	Multi-Status	Risposte multiple (WebDAV)

3xx — Redirection (parte 1)

Codice	Significato	Descrizione breve
300	Multiple Choices	Più opzioni disponibili
301	Moved Permanently	Risorsa spostata in modo permanente
302	Found	Risorsa trovata (redirezione temporanea)
303	See Other	Vedere un'altra risorsa
304	Not Modified	Risorsa non modificata

3xx — Redirection (parte 2)

Codice	Significato	Descrizione breve
305	Use Proxy	Usare un proxy
306	(Unused)	Codice non utilizzato
307	Temporary Redirect	Redirezione temporanea
308	Permanent Redirect	Redirezione permanente

4xx — Client Error (parte 1)

Codice	Significato	Descrizione breve
400	Bad Request	Richiesta non valida
401	Unauthorized	Non autorizzato
402	Payment Required	Pagamento richiesto (non usato)
403	Forbidden	Accesso negato
404	Not Found	Risorsa non trovata
405	Method Not Allowed	Metodo non consentito
406	Not Acceptable	Risorsa non accettabile
407	Proxy Authentication Required	Autenticazione proxy richiesta

4xx — Client Error (parte 2)

Codice	Significato	Descrizione breve
408	Request Timeout	Timeout della richiesta
409	Conflict	Conflitto nella richiesta
410	Gone	Risorsa non più disponibile
411	Length Required	Lunghezza richiesta
412	Precondition Failed	Precondizione fallita
413	Payload Too Large	Corpo richiesta troppo grande
414	URI Too Long	URI troppo lungo
415	Unsupported Media Type	Tipo di media non supportato

4xx — Client Error (parte 3)

Codice	Significato	Descrizione breve
416	Range Not Satisfiable	Range non soddisfacibile
417	Expectation Failed	Aspettativa fallita
418	I'm a teapot	Sono una teiera (scherzoso)
421	Misdirected Request	Richiesta mal indirizzata
422	Unprocessable Entity	Entità non processabile (WebDAV)
423	Locked	Risorsa bloccata (WebDAV)
424	Failed Dependency	Dipendenza fallita (WebDAV)
425	Too Early	Troppo presto

4xx — Client Error (parte 4)

Codice	Significato	Descrizione breve
426	Upgrade Required	Aggiornamento richiesto
428	Precondition Required	Precondizione richiesta
429	Too Many Requests	Troppe richieste
431	Request Header Fields Too Large	Header troppo grande
451	Unavailable For Legal Reasons	Non disponibile per motivi legali

5xx — Server Error (parte 1)

Codice	Significato	Descrizione breve
500	Internal Server Error	Errore interno del server
501	Not Implemented	Funzionalità non implementata
502	Bad Gateway	Gateway non valido
503	Service Unavailable	Servizio non disponibile
504	Gateway Timeout	Timeout del gateway
505	HTTP Version Not Supported	Versione HTTP non supportata

5xx — Server Error (parte 2)

Codice	Significato	Descrizione breve
506	Variant Also Negotiates	Variante negozia anche
507	Insufficient Storage	Spazio insufficiente (WebDAV)
508	Loop Detected	Loop rilevato (WebDAV)
510	Not Extended	Estensione richiesta
511	Network Authentication Required	Autenticazione di rete richiesta

Esercizio: Ottenere risposte 404 e 500 con Postman

1. Ottenere una risposta **404 Not Found**

- Apri Postman e crea una nuova richiesta **GET**.
- Inserisci un URL che non esiste, ad esempio:

```
https://jsonplaceholder.typicode.com/nonexistent
```

- Premi **Send**.

Esempio di risposta:

```
GET /nonexistent HTTP/1.1  
Host: jsonplaceholder.typicode.com
```

```
HTTP/1.1 404 Not Found  
Content-Type: application/json; charset=utf-8  
...
```

```
{  
  "error": "Not Found"  
}
```

2. Ottenere una risposta **500 Internal Server Error**

- Per simulare un errore 500, puoi usare servizi di test come <https://mock.httpstatus.io/>.
- Crea una richiesta **GET** su:
`https://mock.httpstatus.io/500`
- Premi **Send**.

Esempio di risposta:

```
GET /500 HTTP/1.1
Host: httpstat.us

HTTP/1.1 500 Internal Server Error
Content-Type: text/html; charset=utf-8
...

500 Internal Server Error
```


Nota:

Le risposte 404 e 500 sono utili per testare la gestione degli errori nelle applicazioni web.

HTTP vs HTTPS

- **HTTP:** dati trasmessi in chiaro, non sicuro
- **HTTPS:** HTTP su TLS/SSL, dati cifrati e autenticati

HTTPS è lo standard attuale per la sicurezza delle comunicazioni web

Approfondimento: HTTPS e SSL/TLS

- **HTTPS** (HyperText Transfer Protocol Secure) è la versione sicura di HTTP.
- Utilizza **SSL/TLS** (Secure Sockets Layer / Transport Layer Security) per cifrare i dati trasmessi tra client e server.
- La cifratura protegge da intercettazioni e manomissioni dei dati (es. password, dati personali).

Approfondimento: HTTPS e SSL/TLS (2)

- HTTPS garantisce:
 - **Confidenzialità**: i dati non possono essere letti da terzi.
 - **Integrità**: i dati non possono essere modificati durante il transito.
 - **Autenticità**: il client può verificare l'identità del server tramite certificati digitali.
- I certificati SSL/TLS sono rilasciati da **Certificate Authority** (CA) affidabili.
- Oggi HTTPS è fondamentale per la sicurezza di qualsiasi sito web.

Schema: Certificato HTTPS e Certificate Authority (CA)

```
sequenceDiagram
    participant Browser
    participant WebServer
    participant CA as "Certificate Authority"

    Browser->>WebServer: Richiesta HTTPS (Client Hello)
    WebServer->>Browser: Invio certificato SSL/TLS
    Browser->>CA: Verifica certificato (firma digitale)
    CA-->>Browser: Certificato valido (o errore)
    Browser->>WebServer: Scambio chiavi e negoziazione TLS
    WebServer-->>Browser: Connessione cifrata stabilita
    Browser->>WebServer: Richiesta HTTP sicura (GET/POST...)
    WebServer-->>Browser: Risposta HTTP cifrata
```

Evoluzione dei browser e delle tecnologie web

- 1993: Mosaic, primo browser grafico
- 1994: Netscape Navigator
- 1995: Internet Explorer
- 2000: Firefox, Opera, Safari
- 2008: Google Chrome
- Oggi: browser moderni supportano HTML5, CSS3, JavaScript avanzato, WebAssembly
- Le tecnologie web si sono evolute: da pagine statiche a web app dinamiche e interattive

Architettura a livelli: presentazione, logica, dati

- **Presentazione:** interfaccia utente (HTML, CSS, JS)
- **Logica:** elaborazione dati, regole di business (JavaScript lato client, Node.js lato server, PHP, ...)
- **Dati:** archiviazione e gestione delle informazioni (database, file, API)
- Separazione dei livelli = maggiore manutenibilità e scalabilità

Domande di verifica

1. Chi è considerato l'inventore del World Wide Web?

- ☐ Bill Gates
- ☐ Steve Jobs
- ☐ Tim Berners-Lee
- ☐ Mark Zuckerberg

2. Quale dei seguenti protocolli opera al livello 7 (Applicazione) del modello OSI?

- ☐ TCP
- ☐ IP
- ☐ HTTP
- ☐ Ethernet

3. Qual è la principale differenza tra HTTP e HTTPS?

- ☐ HTTPS è più veloce di HTTP
- ☐ HTTP utilizza la porta 443
- ☐ HTTPS cifra i dati trasmessi
- ☐ HTTP non usa certificati digitali

4. Quale status code HTTP indica che una risorsa non è stata trovata?

- ☐ 200
- ☐ 404
- ☐ 500
- ☐ 301

5. Quale strumento è comunemente usato per testare le API web?

- ☐ Photoshop
- ☐ Excel
- ☐ Postman
- ☐ WordPress

Domande aperte

1. Spiega brevemente cosa si intende per "stateless" nel protocollo HTTP.

2. Descrivi i principali vantaggi dell'utilizzo di HTTPS rispetto a HTTP.

3. Riassumi la funzione dei livelli della pila ISO/OSI e indica dove si colloca HTTP.

4. Fai un esempio pratico di richiesta e risposta HTTP, indicando i principali elementi coinvolti.

5. Perché è importante separare la presentazione, la logica e i dati nell'architettura di un'applicazione web?

Soluzioni alle domande di verifica

Risposte a scelta multipla

1. Tim Berners-Lee
2. HTTP
3. HTTPS cifra i dati trasmessi
4. 404
5. Postman

Risposte alle domande aperte

1. HTTP è stateless perché ogni richiesta è indipendente e il server non mantiene informazioni sullo stato tra richieste.
2. HTTPS protegge i dati tramite cifratura, garantisce integrità e autenticità grazie ai certificati digitali.
3. La pila ISO/OSI ha 7 livelli che gestiscono la comunicazione di rete; HTTP si trova al livello Applicazione (7).
4. Esempio: il client invia una richiesta `GET /index.html`, il server risponde con `HTTP/1.1 200 OK`, header e body.
5. Separare presentazione, logica e dati rende l'applicazione più manutenibile, scalabile e modulare.